Introduktion till C++

Uppgift 4.2 – Diablo – Loot

Uppgiftens målsättning

Den studerande ska få bygga på komplexiteten på tidigare spel för att lära sig jobba iterativt.

Uppgift

Uppgiften är att öka komplexiteten på diablo-spelet genom att introducera flertalet olika spelobjekt.

Features

Grundkraven för uppgiften är att följande fungerar:

- Samtlig funktionalitet från 4.1 ska finnas kvar
- Fiender ska hindra spelaren från att ta föremål, besvärjelser och öppna kistor om dem lever.
- Spelaren ska kunna plocka upp föremål
- Spelaren ska kunna få bonusar av föremål
- Spelaren ska kunna öppna kistor och plocka upp föremål från dem
- Spelaren ska kunna läsa besvärjelser och få tidsbegränsade bonusar av dem
- Man ska kunna visa spelarens egenskaper och upplockade objekt (inventory)

Spelaren

- Alla egenskaper, både grund- och sekundära, ska gå att modifiera med föremål och besvärjelser.
- En modifikation adderar ett värde till en egenskap och det går bra att addera negativa värden för att sänka värdet för en egenskap.
- Man ska kunna se spelarens egenskaper *efter* det att alla modifikationer från föremål och besvärjelser har applicerats.
- Man ska kunna välja att se alla objekt som spelaren har plockat upp.

Fiender

- När en fiende dör så finns det en chans att den tappar ett objekt i rummet.

Rum

I rummen ska det nu kunna finnas:

- Kistor
- Föremål
- Besvärjelser

Det ska dyka upp nya alternativ när man är i rummet där man kan välja att öppna kistor, plocka föremål eller använda/plocka upp besvärjelser.

Föremål

Det ska finnas föremål i spelet.

- Spelaren får dessa genom att plocka upp dem från golvet.
- Det kan ligga föremål på golvet när man kommer in i ett rum.
- Det kan även trilla ut föremål från en kista när den öppnas, men det går även bra att objekten plockas upp direkt ur kistan.
- Fiender kan tappa föremål när dem dör.
- När spelaren bär på föremål ska dessa kunna ge bonusar till spelarens egenskaper.
- Föremål ska kunna ge bonusar till både grundegenskaper och sekundära egenskaper.
- Varje föremål har en vikt. Om föremålens totalvikt överstiger spelarens bärkapacitet så kan inga fler objekt plockas upp.

Kistor

- Det kan finnas kistor i rummen.
- Spelaren kan då välja från menyn i rummet att öppna en kista.
- När en kista öppnas så trillar alla föremål ut på golvet, alternativ lösning är att man kan plocka upp dom direkt ur kistan.

Besvärjelser

- Det kan finnas besvärjelser (spells) i rummen.
- Spelaren kan då välja från menyn i rummet att plocka upp dem, och de aktiveras då automatiskt.

- När dem aktiveras ger dom bonus till spelaren men under en tidsbegränsning. Denna tid kan mätas exempelvis i antal rum man besöker eller antal handlingar man utför.

Valfria tillägg:

Följande punkter är valfria tillägg för lite extra utmaning:

- En modifikation av en grundegenskap räknas in när de sekundära egenskaperna som använder den grundegenskapen beräknas.
  Exempel: Vi plockar upp en rustning som ökar spelarens smidighet med 2 och attackvärdet med 4.
  Attackvärdet måste då räknas ut från grunden, och addera den nya bonusen på fyra (styrka * (smidighet + 2)) + 4.

- En kista kan precis som dörrarna vara låst men alla kistor behöver inte vara låsta.

- Spelaren kan kasta valfria items som man bär på, som då hamnar på golvet i rummet.

Inlämning

Gör mapp med uppgiftens namn. I denna mapp skapar ni ytterligare tvåmappar. En som heter **Exe** och en som heter **Source**.

I **Source**-mappen innehåller all källkod samt projekt filer.
Exe-mappen ska endast filer som behövs för att köra programmet ligga.
Ta bort alla skräpfiler!!

Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.

Verktyg

För att lösa uppgiften är följande verktyg, eller koncept tillåtna. En del av er kanske redan har kunskaper i C++ och vet andra sätt att lösa uppgiften med hjälp av andra koncept. Det är inte tillåtet, uppgiften kan och skall lösas endast med angivna verktyg!

```
#include, #pragma once
assert
funktioner
namespace
char, short, int, long, float, double, bool, void, unsigned
std::string, std::string_view, std::vector, std::bitset, iteratorer
class, struct, public, private
const, static
enum class
std::array, referenser, pekare, std::unique_ptr<>, std::span<T>
std::shared_ptr<>, std::weak_ptr<>, std::make_unique<>,
std::make_shared<>
typedef
if, else, else if, switch, while, do, for, return, break
random_device, mt19937, uniform_int_distribution
cout, cin
static_cast<>, const_cast<>
system(“cls”), system(“pause”)
```